📓 جزوه سیستم عامل - فصل ۶: ابزار همگامی (بخش پنجم و پایانی)

(عاقبت‌به‌خیری، بن‌بست و خلاصه فصل)

۱۶. عاقبت‌به‌خیری (Liveness)

تا اینجا ابزارهایی (مثل Mutex و Semaphore) ساختیم تا "انحصار متقابل" را تضمین کنیم (یعنی دو نفر همزمان وارد نشوند). اما اگر سیستم امن باشد ولی "کار نکند" چه فایده‌ای دارد؟

- **تعریف Liveness (عاقبت‌به‌خیری):** مجموعه‌ای از ویژگی‌ها که تضمین می‌کند پردازه‌ها در طول اجرایشان بالاخره پیشرفت می‌کنند و متوقف نمی‌شوند.
- **شکست در Liveness:** یعنی وقتی پردازه در یک وضعیت گیر می‌کند (مثل حلقه بی‌نهایت).

ما دو نوع شکست مهم در Liveness داریم:

1. **بن‌بست (Deadlock)**
2. **وارونگی اولویت (Priority Inversion)**

۱۷. بن‌بست (Deadlock)

وضعیتی است که دو یا چند پردازه منتظر رویدادی هستند که فقط توسط یکی دیگر از همان پردازه‌های منتظر می‌تواند ایجاد شود. (همه منتظر همدیگرند).

مثال کلاسیک بن‌بست (با دو سمافور S و Q):

فرض کنید دو پردازه P_0 و P_1 داریم و دو سمافور با مقدار اولیه ۱.

- **پردازه P_0:** اول wait(S) را می‌زند (S را می‌گیرد)، حالا می‌خواهد wait(Q) را بزند.
- **پردازه P_1:** اول wait(Q) را می‌زند (Q را می‌گیرد)، حالا می‌خواهد wait(S) را بزند.

سناریوی فاجعه:

1. P_0 سمافور S را می‌گیرد.

2. (سوئیچ پردازنده)
3. 1_P سمافور Q را می‌گیرد.
4. حالا 0_P منتظر Q است (که دست 1_P است).
5. و 1_P منتظر S است (که دست 0_P است). **نتیجه:** هر دو تا ابد منتظر می‌مانند! هیچ‌کس signal نمی‌دهد.

۱۸. وارونگی اولویت (Priority Inversion)

این مشکل زمانی رخ می‌دهد که یک پردازه با **اولویت بالا** مجبور می‌شود منتظر یک پردازه با **اولویت پایین** بماند (انگار جای اولویت‌ها عوض شده!).

سناریوی وارونگی اولویت:

سه پردازه داریم با اولویت‌های: **L (کم) < M (متوسط) < H (زیاد)**

1. پردازه **L** منبعی (مثلاً یک قفل) را می‌گیرد.
2. پردازه **H** (اولویت بالا) درخواست آن منبع را می‌دهد -> چون منبع اشغال است، **H** منتظر **L** می‌ماند.
3. ناگهان پردازه **M** (اولویت متوسط) وارد می‌شود. چون اولویت M از L بیشتر است، پردازنده را از L می‌گیرد (Preempt می‌کند).

نتیجه عجیب:

- پردازه **H** (که خیلی مهم است) منتظر **L** است.
- پردازه **L** نمی‌تواند کار کند چون **M** دارد اجرا می‌شود.
- عملاً **H** دارد منتظر **M** می‌ماند (در حالی که هیچ ربطی به هم ندارند!). این یعنی یک پردازه متوسط، مانع اجرای پردازه مهم شده است.

✓ راه حل: پروتکل ارث‌بری اولویت (Priority Inheritance Protocol)

طبق این پروتکل، اگر پردازه **H** منتظر منبعی باشد که دست **L** است، پردازه **L** موقتاً **اولویت H را به ارث می‌برد** (اولویتش بالا می‌رود).

- با این کار، پردازه **M** دیگر نمی‌تواند مزاحم **L** شود.

- پردازه L سریع کارش را تمام می‌کند، منبع را آزاد می‌کند و اولویتش به حالت قبل برمی‌گردد.
- حالا H منبع را می‌گیرد و اجرا می‌شود.

(نکته جالب: این مشکل واقعاً در مریخ‌نورد Pathfinder ناسا رخ داد و سیستم مدام ریست می‌شد تا اینکه با همین پروتکل مشکل را حل کردند!)

۱۹. ارزیابی ابزارهای همگامی (چه زمانی از چه چیزی استفاده کنیم؟)

در فصل ۶ ابزارهای مختلفی یاد گرفتیم. کدام بهتر است؟ بستگی به میزان **رقابت (Contention)** دارد.

مقایسه CAS (سخت‌افزاری) و Mutex (نرم‌افزاری):

1. **رویکرد CAS (خوش‌بینانه):** فرض می‌کند تداخل کم است. متغیر را آپدیت می‌کند، اگر نشد دوباره تلاش می‌کند.
2. **رویکرد Mutex (بدبینانه):** فرض می‌کند تداخل زیاد است. اول قفل می‌کند، بعد وارد می‌شود.

نمودار کارایی (نکته امتحانی):

- **رقابت پایین (Low Contention):** CAS سریع‌تر است (سربار قفل ندارد).
- **رقابت متوسط:** CAS بسیار بهتر عمل می‌کند.
- **رقابت بسیار بالا (High Contention):** ابزارهای سنتی (Mutex/Semaphore) بهترند. (چون در CAS پردازنده مدام در حال تلاش مجدد و درجا زدن است، اما در Mutex پردازه‌های منتظر به خواب می‌روند).

۲۰. خلاصه کل فصل ۶ (جمع‌بندی شب امتحان)

اگر می‌خواهید کل این فصل را در ۵ خط مرور کنید:

1. **شرط مسابقه (Race Condition):** وقتی خروجی برنامه به شانس و زمان‌بندی بستگی دارد (بد است).

2. **بخش بحرانی (CS):** جایی که متغیر مشترک تغییر می‌کند. باید ۳ شرط داشته باشد: **انحصار متقابل**، **پیشروی**، **انتظار محدود**.
3. **Mutex:** ساده‌ترین قفل. توابع acquire و release. (مناسب برای انحصار متقابل ساده).
4. **Semaphore:** متغیر صحیحی که با wait و signal کار می‌کند. (مناسب برای همگامی و مدیریت منابع محدود).
5. **Monitor:** کلاس سطح بالا که خودش مدیریت قفل را انجام می‌دهد و امن‌تر است. (جلوگیری از خطای برنامه‌نویس).
6. **Liveness:** باید مراقب **بن‌بست** (قفل شدن همه) و **وارونگی اولویت** (معطل شدن پردازه مهم توسط پردازه معمولی) باشیم.

پایان فصل ۶ - موفق باشید! (۲۰ بگیرید!) ☺